

Le storefront est intégralement branché sur l'API existante : catalogue, personnalisation, upload, commande, tracking, mémoire QR et authentification. Les points ci-dessous sont les **seuls endroits où le frontend a dû se débrouiller sans endpoint**. Aucun n'a été comblé par des données fictives : chacun est marqué // MISSING API : dans le code et signalé honnêtement dans l'interface. Priorités classées par impact client réel.

PRIO	MANQUE	CONTOURNEMENT ACTUEL CÔTÉ FRONTEND
P1	Devis de commande	Frais de livraison recalculés dans le front — seule duplication de règle métier
P1	Historique client	localStorage du navigateur + relecture via tracking public
P1	Liste des catégories	Dérivée d'un GET /api/products?size=100
P2	Commande par id (client)	OrderResponse transporté dans le state du routeur
P2	Mise à jour du profil	Champs en lecture seule
P2	Déconnexion / révocation	Suppression locale des tokens
P3	Avis & notes	Section supprimée (les faux témoignages ont été retirés)
P3	Alerte réapprovisionnement	Aucun : bouton désactivé, sans promesse

PRIORITÉ 1 — BLOQUANT POUR LA QUALITÉ DU PARCOURS D'ACHAT

1 Devis de commande (order quote)

C'est le plus important. Aujourd'hui POST /api/orders est le seul code qui sait chiffrer un panier — et il le fait *en créant la commande*. Un client en paiement à la livraison ne peut donc pas voir le montant qu'on lui réclamera à la porte avant de valider. Le frontend réplique donc calculateDeliveryFee (30,00 MAD, offerts à partir de 500,00 MAD) dans lib/delivery.js, affiché partout comme « estimation ». **C'est la seule règle métier dupliquée dans le front**, et elle divergera dès que giftme.order.* changera.

POST /api/orders/quote (public, sans effet de bord)

REQUÊTE { items: [{ productId, quantity }] } // sous-ensemble de CreateOrderRequest

RÉPONSE { subtotal, deliveryFee, total, freeDeliveryThreshold,
items: [{ productId, unitPrice, quantity, totalPrice }],
unavailable: [{ productId, reason }] }

CLÉ Aucune réservation de stock, aucune écriture. unavailable permet au panier de signaler un produit désactivé ou en rupture avant le checkout.

Réutilisable : extraire la boucle de valorisation de OrderServiceImpl.addOrderItem (sans decreaseStock) + calculateDeliveryFee tel quel. Estimation : ~1 service, ~1 DTO.

2 Historique de commandes du client connecté

Order.user est déjà renseigné quand un token CUSTOMER accompagne POST /api/orders — mais **rien ne permet de relire ces commandes** : /api/admin/orders/** exige ROLE_ADMIN et le tracking public est indexé par code. La page Compte liste donc les commandes passées *depuis ce navigateur* (localStorage), puis relit l'état réel de chacune via GET /api/tracking/{code}. Conséquence assumée et affichée à l'utilisateur : cache vidé ou second appareil = historique vide.

GET /api/orders/me?page=&size=&sort= (authentifié, rôle CUSTOMER)

RÉPONSE PagedResponse<OrderSummaryResponse>

FILTRAGE where order.user.id = principal.id // jamais un id passé en paramètre

Réutilisable : AdminOrderSummaryResponse convient champ pour champ (tout y appartient déjà au client) — le renommer OrderSummaryResponse et le partager. La pagination existe déjà dans AdminOrderServiceImpl.search.

3 Liste des catégories

category est un simple VARCHAR(100) sur products, sans table ni endpoint. Pour afficher les filtres de la boutique et les liens du pied de page, le front charge **100 produits juste pour en extraire les valeurs distinctes**. Cela fonctionne avec 3 produits ; cela cassera silencieusement dès que le catalogue dépassera une page.

GET /api/categories (public)

RÉPONSE [{ name, productCount }] // produits actifs uniquement

NOTE Un SELECT category, count(*) ... WHERE active GROUP BY category suffit ; l'index idx_products_category existe déjà. Une vraie table Category (avec libellé, ordre d'affichage, image) serait préférable à terme.

PRIORITÉ 2 — DETTE FONCTIONNELLE VISIBLE

GET /api/orders/{id}

authentifié, scopé au propriétaire

La page de confirmation reçoit l'OrderResponse via le state du routeur : un rafraîchissement ou un lien ouvert directement **perd la commande**, et le front affiche alors un écran de repli vers le tracking. Impossible de faire mieux sans un endpoint de relecture. Renvoyer OrderResponse existant, avec `order.user.id = principal.id` (404 sinon, pas 403 — ne pas révéler l'existence).

PUT /api/auth/me

authentifié

Aucun moyen de corriger un nom, un téléphone ou un email. La page Compte affiche donc les champs en lecture seule. Requête : `{ fullName, phone }` (email et mot de passe méritent chacun leur flux vérifié). Rappel : Customer est dédoublé par **téléphone**, pas par email — décider si une mise à jour du profil doit propager sur la fiche client.

POST /api/auth/logout

authentifié

Les refresh tokens ne sont révoqués que par rotation. Se déconnecter ne fait donc que vider le stockage local : l'access token reste valable jusqu'à 15 minutes et le refresh token 14 jours s'il a fuité. Sur un poste partagé, ce n'est pas une vraie déconnexion. Un simple `delete from refresh_tokens where token_hash = ...` suffit.

PRIORITÉ 3 — NOUVELLES FONCTIONNALITÉS

Ressource Avis / Notes

aucune entité, table ni endpoint

La page d'accueil affichait **quatre chiffres inventés** (« 10K+ cadeaux livrés », « 4,8/5 ») et **trois faux témoignages nominatifs**. Tout a été retiré : publier des chiffres commerciaux non vérifiés est un risque réglementaire, pas un sujet de rédaction. Remplacés par quatre promesses vérifiables issues du backend lui-même. Pour restaurer une vraie preuve sociale : GET /api/products/{id}/reviews + POST d'un avis consenti rattaché à une commande DELIVERED (seule façon d'éviter les avis non authentiques), avec modération admin.

Alerte réapprovisionnement

produit en rupture

Quand `stock = 0`, le front désactive l'ajout au panier et n'offre rien d'autre — plutôt qu'un bouton « prévenez-moi » qui ne préviendrait personne. POST /api/products/{id}/notify-me { email } le rendrait réel.

⚠ Ce n'est pas un manque, mais une décision à trancher

GET /api/products/{id} et /slug/{slug} ne filtrent pas sur **active**. ProductServiceImpl.list() passe bien `active = true` à sa Specification, mais les deux accès unitaires sont de simples findById / findBySlug. Vérifié sur le backend en cours d'exécution : après désactivation, /api/products/slug/qr-memory-experience répond 200 avec `active: false`.

Le README annonce « active products only » pour le catalogue public ; cette mention ne vaut que pour la liste. Un commentaire dans le code confirme que c'est intentionnel (« inactive products are still reachable by direct id/slug lookup »), mais le comportement est piégeux : un produit retiré restait affiché « En stock » avec un bouton d'ajout actif tant que le front ne lisait pas `active` lui-même. C'est corrigé côté frontend (page produit et panier vérifient le drapeau).

À décider : soit renvoyer 404 aux appels publics sur un produit inactif et laisser /api/admin/products/{id} servir l'admin, soit conserver le comportement et **le documenter explicitement dans le README**. Dans les deux cas, préciser aussi que Jackson est configuré en `non_null` : les champs nuls sont *absents* du JSON, pas à null — ce qui compte pour tout consommateur (mainImage n'apparaît jamais pour le catalogue initial).